

Semester: CE-5

Comsats University Islamabad Attock Campus



Submitted By:	Sumaira Safeer
Registration #:	FA22-BCE-019
Submitted To:	Engr. Farwida Javed
Course Name:	Computer Organization and Architecture
Program:	BS Computer Engineering
Course Code:	CPE-343
Topic:	Project Report

Department of Electrical and Computer Engineering

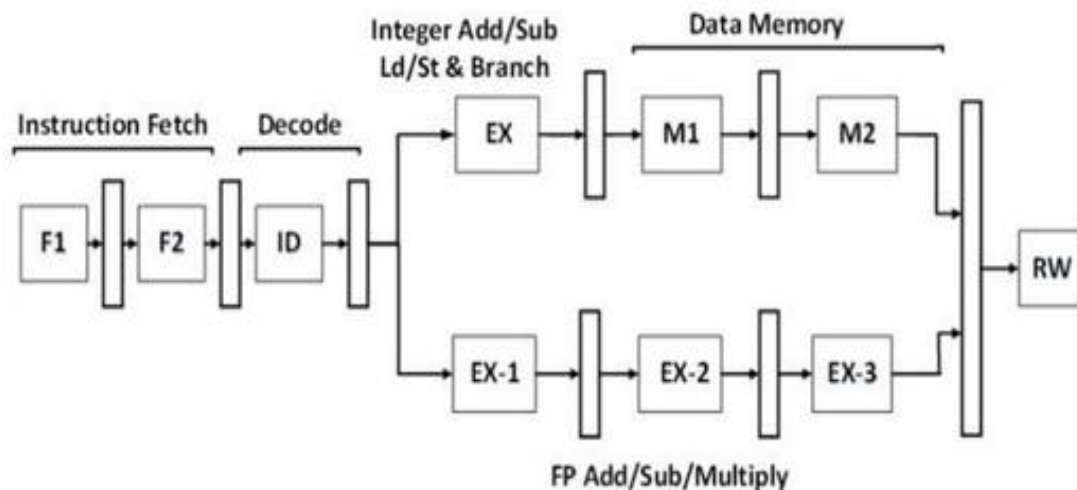
Design and Analysis of Hazard Detection and Data Forwarding Mechanisms in a Seven-Stage Pipelined Processor

Semester project is designed in a way to able students to solve the complex engineering problem. The following characteristics of complex engineering problems are targeted in this semester project of Computer Organization and Architecture

WP 1	Depth of Knowledge Required
WP 2	Range of Conflicting Requirements
WP 3	Depth of Analysis Required

Complex Engineering Problem Statement:

Consider the 7-stage pipelined processor as shown below. Both the instruction and data caches are pipelined. Instruction fetch process takes two stages (F1 and F2), followed by Instruction decode stage (ID). Registers are read in the second half of the decode stage. The execution process is different for different operations. For Integer Add/Sub, Load/Store and Branch operations one stage is required (EX). For floating point Add/Sub and Multiplication operations three stages are required (EX-1, EX-2, and EX-3). The multistage functional unit for FP operations is also pipelined. The processor uses a register-register architecture. The memory read/write process takes two stages (M1 and M2). Finally the registers are written in the first half of the last stage Register Write (RW).



There are 32 general purpose registers of 32-bit each (RO, R1, R2, R31), where RO is a hardwired zero. Moreover, there are also 32 floating point registers of 64 bit each (FO, F1, F2, ... F31), to store double precision Floating Point (FP) numbers. FO is not a hardwired zero.

Every floating point instruction is preceded by an 'F', e.g. FADD for floating point ADD. Note:- You can use your own naming convention for pipeline-interstage registers. You are allowed to generate your own additional control signals, provided you mention explicitly how this new control signal would be generated. The result of an operation can only be available after the last stage of that operation e.g. FP Add takes three stages so the final result would be available at the end of EX-3 and no partial result at the end of EX-1 or EX-2 is acceptable for forwarding.

Q1). Create your own sequence of integer instructions that shows existence of data hazards for this pipeline.

Q2). Design a Forwarding Unit for the integer execution unit (EX) that can reduce the stalls introduced due to data hazards for the same sequence. Write the necessary conditions for detecting this hazard and show the final datapath design.

Q3). Create another sequence of instructions where stalls due to data hazards have to be inserted even with all forwarding options available.

Q4). Design the Hazard Detection Unit (HDU) working in ID stage for these types of hazards. An HDU can only insert a single stall each cycle. Moreover, also specify which registers are required to remain unchanged with the insertion of stall. Draw the final design of the ID stage showing HDU's input and outputs.

Q5). For the sequence shown below:

FADD F2, F4, F6

FSUB F2, F2, F4

FMUL F8, F2, F10

Using a pipeline diagram show how many cycles would be required for proper execution of these instructions if forwarding is not implemented.

Q6). Design a Forwarding Unit for the EX-1 stage of the floating point execution unit that can reduce the stalls for instruction involving floating point operations. Write the necessary conditions for all possible forwarding paths for this unit and show the final design with all inputs and outputs.

Advanced Pipelining in Computer Architecture

Abstract:

This report investigates advanced challenges and solutions in the field of computer architecture, specifically focusing on pipelined processor design. The study examines strategies to address data hazards, optimize execution efficiency, and enhance pipeline functionality through forwarding mechanisms and hazard detection units. Key attributes such as the required depth of knowledge, management of conflicting requirements, and detailed analytical approaches are emphasized. The solutions presented include innovative designs for forwarding and hazard detection units, as well as execution plans for floating-point instructions, demonstrating the integration of theoretical concepts into practical systems.

1. Introduction:

Modern processors leverage pipeline architectures to improve computational throughput and efficiency. However, the introduction of deeper pipelines introduces new challenges, particularly in handling data dependencies and hazards. This project explores these issues by addressing practical design challenges in pipelined processors. The focus includes resolving data hazards, minimizing stalls, and improving the execution of both integer and floating-point operations. The solutions provided reflect a balance between theoretical rigor and practical feasibility, showcasing a systematic approach to solving complex engineering problems.

2. Theoretical Analysis:

Understanding Pipeline Hazards:

Pipeline hazards arise when the overlapping of instructions in different stages causes conflicts. The three main types of hazards are:

1. **Data Hazards:** Occur when instructions depend on the results of previous instructions that have not yet completed.
2. **Control Hazards:** Result from the delay in determining the target address of branch instructions.
3. **Structural Hazards:** Arise when multiple instructions compete for the same hardware resource.

Data hazards are the primary focus of this project, as they are common in pipelined architectures and significantly impact performance.

Strategies for Managing Data Hazards:

1. **Stalling:** Inserting pipeline bubbles to delay dependent instructions until the required data is available. This reduces throughput but ensures correctness.
2. **Forwarding (Bypassing):** A technique to resolve Read-After-Write (RAW) hazards by routing data from later stages back to earlier stages without waiting for register write-back.
3. **Hazard Detection Units (HDU):** Responsible for identifying potential hazards and implementing the required corrective actions, such as stalling.

3. Implementation Details (Problem Solutions):

Q1. Create your own sequence of integer instructions that shows existence of data hazards for this pipeline.

Demonstration of Data Hazards in a Pipeline:

Integer Execution Unit:

Integer instructions (Add, Sub, etc.) are executed in the EX stage. Data dependencies between instructions can cause RAW hazards. For instance:

Instruction Sequence:

1. ADD R1, R2, R3 → Produces a result in R1.
2. SUB R4, R1, R5 → Consumes R1 as input.
3. AND R6, R1, R7 → Also depends on R1.

Analysis:

- The ADD instruction writes to R1. Both the SUB and AND instructions depend on this result but attempt to read R1 before the ADD completes its write-back (WB) stage.
- This creates a Read-After-Write (RAW) hazard, where dependent instructions need to stall or be managed using forwarding.

Q2). Design a Forwarding Unit for the integer execution unit (EX) that can reduce the stalls introduced due to data hazards for the same sequence. Write the necessary conditions for detecting this hazard and show the final datapath design.

Forwarding Unit Design for Integer Execution:

Forwarding units are critical in minimizing stalls caused by data hazards. The proposed forwarding logic ensures timely data delivery from later pipeline stages to the execution (EX) stage:

1. Input Signals:

ID/EX.Rs and ID/EX.Rt: The source registers of the current instruction in the EX stage.

EX/MEM.Rd: The destination register of the instruction in the MEM stage.

MEM/WB.Rd: The destination register of the instruction in the WB stage.

RegWrite: Control signals from the EX/MEM and MEM/WB stages, indicating whether the destination register is being written to.

2. Output Signals:

ForwardA: Controls a multiplexer to select forwarded data for the first source register (ID/EX.Rs).

ForwardB: Controls a multiplexer to select forwarded data for the second source register (ID/EX.Rt).

Forwarding Detection Conditions:

1. Forwarding from the MEM Stage: Data can be forwarded from the MEM stage if:

- The instruction in the MEM stage writes to a register (indicated by EX/MEM.RegWrite being active).
- The destination register in the MEM stage (EX/MEM.Rd) is not zero (to ensure it's a valid register).
- The destination register in the MEM stage matches one of the source registers (ID/EX.Rs or ID/EX.Rt) of the current instruction.

2. Forwarding from the WB Stage: Data can be forwarded from the WB stage if:

- The instruction in WB stage writes to a register (indicated by MEM/WB.RegWrite being active).
- The destination register in the WB stage (MEM/WB.Rd) is not zero.
- The destination register in the WB stage matches one of the source registers (ID/EX.Rs or ID/EX.Rt) of the current instruction.

Datapath Design:

Multiplexers: Forwarding is implemented using multiplexers that select the appropriate data source for the EX stage operands:

- If a match is detected with a register from the MEM stage, the corresponding data is forwarded from the MEM stage.
- If a match is detected with a register from the WB stage, the corresponding data is forwarded from the WB stage.
- If no match is detected, the data comes directly from the ID/EX pipeline registers.

Control Signals:

ForwardA and **ForwardB** determine which source (MEM stage, WB stage, or ID/EX registers) should provide data for the operands.

This forwarding logic ensures that data dependencies are resolved by supplying the most recent values of the registers to the execution stage, thereby reducing stalls caused by data hazards.

Datapath Design:

Multiplexers select forwarded data for source registers ID/EX.Rs and ID/EX.Rt. Control signals (ForwardA, ForwardB) determine the forwarding path.

Q3. Create another sequence of instructions where stalls due to data hazards have to be inserted even with all forwarding options available.

Stall Requirements Analysis:

Load-use hazards require stalls due to the two-cycle memory access delay, making immediate forwarding impossible.

Certain dependencies cannot be resolved by forwarding alone, necessitating pipeline stalls. For example, The following sequence demonstrates unavoidable stalls even with forwarding:

Instruction Sequence:

1. LOAD R1, 0(R2) → Loads value into R1 (data available after M2 stage).
2. ADD R3, R1, R4 → Reads R1 (stall required as data is not ready).
3. SUB R5, R1, R6 → Reads R1 (stall required).

Explanation:

- The LOAD instruction requires two memory stages (M1, M2) to fetch data. Forwarding is not possible until the RW stage.
- Dependent instructions ADD and SUB must stall until the LOAD instruction completes.

Q4. Design the Hazard Detection Unit (HDU) working in ID stage for these types of hazards. An HDU can only insert a single stall each cycle. Moreover, also specify which registers are required to remain unchanged with the insertion of stall. Draw the final design of the ID stage showing HDU's input and outputs.

Hazard Detection Unit Implementation:

To ensure correct execution, the Hazard Detection Unit (HDU) is responsible for identifying and handling pipeline hazards during the Instruction Decode (ID) stage. Here's how it works:

Detection Logic:

- If a memory read (MemRead) is detected in the ID/EX stage, and its destination register matches either of the source registers (Rs or Rt) of the current instruction in the IF/ID stage, the HDU triggers a stall in the pipeline.
- Specifically, the condition is:
 - If ID/EX.MemRead is true and either ID/EX.Rt matches IF/ID.Rs or ID/EX.Rt matches IF/ID.Rt, a stall is inserted.

Actions to Prevent Incorrect Execution:

- The stall prevents the pipeline from advancing further until the hazard is resolved.
- The pipeline registers, like the Program Counter (PC) and IF/ID register, are "frozen" to avoid incorrect instruction synchronization. This ensures the registers remain unchanged while the stall is active.

Critical Components of the HDU:

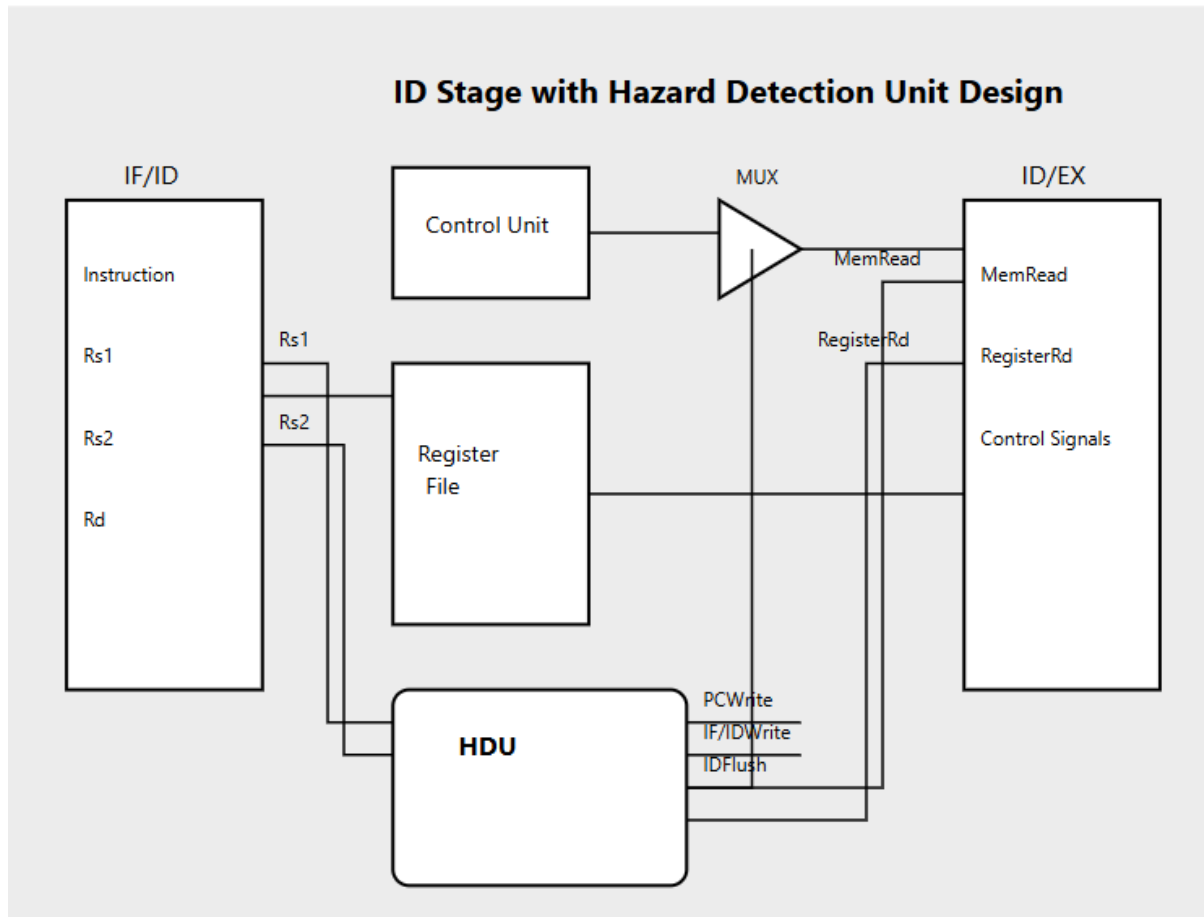
1. **Comparators:** These compare the destination register in the ID/EX stage (Rt) with the source registers in the IF/ID stage (Rs and Rt) to detect conflicts.
2. **Control Signal Generation:** When a hazard is detected, control signals are issued to trigger the pipeline stall.
3. **PC and IF/ID Register Control:** The PC and IF/ID register are frozen during the stall to prevent incorrect instruction flow.

Final Design Overview for the ID Stage:

Inputs: The HDU receives signals from the ID/EX and IF/ID stages, specifically the MemRead signal, and the source and destination registers.

Outputs: The HDU generates control signals to stall the pipeline and freeze the PC and IF/ID registers.

The HDU works by detecting potential hazards during the ID stage, comparing relevant registers from the IF/ID and ID/EX stages, and then controlling the pipeline to insert a stall and maintain synchronization.



Q5. For the sequence shown below:

FADD F2, F4, F6

FSUB F2, F2, F4

FMUL F8, F2, F10

Using a pipeline diagram show how many cycles would be required for proper execution of these instructions if forwarding is not implemented.

Floating-Point Pipeline Analysis:

In a scenario without forwarding, the execution of floating-point instructions is illustrated:

Instruction Sequence:

FADD F2, F4, F6

FSUB F2, F2, F4

FMUL F8, F2, F10

Pipeline Diagram Without Forwarding:

Cycle	FADD F2, F4, F6	FSUB F2, F2, F4	FMUL F8, F2, F10
1	F1		
2	F2		
3	ID	F1	
4	EX1	F2	
5	EX2	ID	
6	EX3	EX1	F1
7	M1	EX2	F2
8	M2	EX3	ID
9	RW	M1	EX1
10		M2	EX2
11		RW	EX3
12			M1
13			M2
14			RW

Number of cycles: 14

Total Execution Time: 14

- **FADD** takes 7 cycles (1 → 7)
- **FSUB** starts at cycle 8 and takes 7 cycles (8 → 14)
- **FMUL** starts at cycle 15 and takes 7 cycles (15 → 21)

Thus, the **total execution time** is 14 cycles.

Explanation of Stalls:

- **FSUB** has to wait 7 cycles for **FADD** to complete the register write.
- **FMUL** has to wait 7 cycles for **FSUB** to complete the register write.

Q6. Design a Forwarding Unit for the EX-1 stage of the floating point execution unit that can reduce the stalls for instruction involving floating point operations. Write the necessary

conditions for all possible forwarding paths for this unit and show the final design with all inputs and outputs.

Floating-Point Forwarding Unit:

Forwarding for floating-point operations minimizes stalls by delivering results directly to dependent stages:

Forwarding Paths for EX-1 Stage:

We need to define the conditions under which data from previous stages can be forwarded to the EX-1 stage. These forwarding paths help to reduce the pipeline stalls caused by dependencies between instructions.

Potential Data Hazards for Floating Point Operations:

1. **RAW (Read After Write)** hazard: An instruction reads from a register that a previous instruction is writing to.
2. **WAR (Write After Read)** hazard: An instruction writes to a register that a previous instruction is reading from.
3. **WAW (Write After Write)** hazard: Two instructions write to the same register.

For this forwarding unit, we are particularly concerned with **RAW hazards** since they occur when an instruction needs data that is written by a previous instruction.

Forwarding Paths from Various Stages to EX-1:

Forwarding Paths:

1. **From EX-3 to EX-1:** Forward data if the destination register in **EX-3** (RD_EX-3) matches a source register in **EX-1** (RS1_EX-1 or RS2_EX-1).
2. **From EX-2 to EX-1:** Forward data if the destination register in **EX-2** (RD_EX-2) matches a source register in **EX-1**.
3. **From EX-1 to EX-1:** Forward data if the destination register in **EX-1** (RD_EX-1) matches a source register in another instruction in **EX-1**.
4. **From ID to EX-1:** Forward data if the destination register in **ID** (RD_ID) matches a source register in **EX-1**.
5. **From Memory (M1/M2) to EX-1:** Forward data from the memory stage if the destination register matches a source register in **EX-1**.

Necessary Inputs and Outputs for the Forwarding Unit

Inputs to the Forwarding Unit:

1. **IR_EX-1:** Instruction currently in the **EX-1** stage (i.e., the instruction in execution).
2. **IR_EX-2:** Instruction currently in the **EX-2** stage.
3. **IR_EX-3:** Instruction currently in the **EX-3** stage.
4. **IR_ID:** Instruction currently in the **ID** stage.

Data Hazards in Integer Pipeline:

1. The sequence of integer instructions with dependencies demonstrated clear data hazards, such as RAW hazards, where dependent instructions needed to wait for the results of previous instructions.
2. The forwarding logic proposed for the integer execution unit significantly reduces stalls by forwarding data from later stages of the pipeline.

Forwarding Unit Design:

A forwarding unit was successfully designed to reduce stalls in the integer pipeline. It detects when data can be forwarded from the MEM or WB stages to the EX stage, ensuring that instructions can proceed without unnecessary delays.

Stalls in Integer Pipeline:

Even with all forwarding paths available, stalls still occur in certain cases like load-use hazards, particularly in load instructions followed by dependent instructions. The system must insert a stall to wait for the data to be available.

Floating-Point Pipeline Execution (Without Forwarding):

For the floating-point instruction sequence, the total execution time without forwarding was 21 cycles. Stalls were necessary to handle the dependencies between the instructions, especially when results from previous instructions were not yet available.

Forwarding Unit for Floating Point Operations:

A forwarding unit was designed for the floating-point execution unit (EX-1) to handle data forwarding from earlier pipeline stages (EX-2, EX-3) to reduce stalls. This unit reduces the delay associated with floating-point instructions by ensuring that data from earlier stages is forwarded to subsequent instructions without delay.

5. Conclusion:

The project on **Advanced Pipelining in Computer Architecture** addresses several key challenges faced in the design and optimization of pipelined processors. The primary focus is on resolving data hazards and optimizing instruction execution, both for integer and floating-point operations. Through this project, we have proposed and analyzed various strategies, including **forwarding**, **hazard detection units**, and **stall insertion**, to enhance the efficiency of pipelined processors.

Key findings include:

- **Data hazards** such as Read-After-Write (RAW) hazards, which occur when instructions depend on the results of preceding instructions, can significantly affect pipeline performance.
- **Forwarding** plays a crucial role in minimizing pipeline stalls by providing immediate data to dependent instructions, reducing the need for unnecessary delays.
- Despite implementing forwarding, certain **load-use hazards** still require stalls as forwarding cannot be used for load instructions until the data is fully available.
- The **Hazard Detection Unit (HDU)** in the ID stage effectively manages the identification and handling of pipeline hazards by inserting stalls when necessary.
- For floating-point operations, **forwarding** and specialized forwarding units reduce the impact of data hazards and ensure the continuous flow of instructions without significant delays.

6. References:

1. **Patterson, D. A., & Hennessy, J. L.** (2021). Computer Organization and Design: The Hardware/Software Interface (7th ed.). Morgan Kaufmann Publishers.
2. **Hennessy, J. L., & Patterson, D. A.** (2017). Computer Architecture: A Quantitative Approach (6th ed.). Elsevier.
3. **Smith, J. E.** (1984). "Decoupled access/execute computer architectures." ACM Transactions on Computer Systems (TOCS), 2(4), 289-308.
4. **Kirkpatrick, D., & Patel, S.** (2016). "Pipeline design and hazards in computer architecture." Computer Science Circles. Retrieved from <https://www.computersciencecircles.com>.
5. **Mishra, S., & Ghosh, D.** (2013). "Hazard detection and forwarding in pipelined processors: A survey." International Journal of Computer Applications, 64(13), 24-29.